

Conversation transcript: SA crash risk data ingestion pipeline

Participants	Aftab Hussaini (student) and Claude (Anthropic AI assistant; configured model <code>claude-opus-5-5</code>)
Date	Saturday 10 October 2026 (Adelaide local time)
What is included	Every message you sent and every reply Claude gave, word for word. Claude's tool use (commands run, files created or edited, web pages read) is listed as one line per action. Claude's internal reasoning is not included. Attached files are shown by name only; the code itself is in <code>sa_crash_pipeline.zip</code> .

You — Sat 26 Sept 2026, 13:28

don't write my code for me but if i were to create an automated pipeline for an ai system that utilises machine learning to address hybrid crash risk and severity prediction at South Australian Intersections and Road Segments. give me some suggestions on how i should approach the fetch, validate, transform and load processes.

Claude — 13:29

Actions taken

- Web search: "data.sa.gov.au road crash data South Australia dataset"

Here's how I'd think about each stage. The main point is that "hybrid" means two different units of analysis, frequency per site and severity per crash, and that choice affects all four stages.

Fetch

Sources to plan for. The core is DIT's crash data on data.sa.gov.au. It is published by the Department for Infrastructure and Transport under CC BY 3.0 AU, and there is a separate crash locations dataset covering sites from 2008 onward. It usually comes as related tables (crash, units, casualties) joined on a report ID. Fetch these as separate entities, not one flat file. For a risk model you'll also need:

- **Exposure:** DIT traffic volumes (AADT). Without a denominator you're only modelling "where traffic is", not risk.
- **Road network geometry:** for intersection nodes and segments.
- **Speed zones and road attributes:** lanes, divided or undivided, signalisation.
- **Context:** BoM weather, and optionally ABS boundaries for area-level covariates.

Mechanics.

- The SA portal runs on CKAN. Poll the package metadata, such as the last-modified timestamps and resource hashes, and only download when something has changed. Avoid scraping on a timer.
- Releases are roughly annual, so a metadata-change sensor fits better than a fixed cron schedule.
- Land every download untouched in an immutable raw zone, tagged with release date and checksum. DIT revises past records: a crash's severity can be upgraded if someone dies within 30 days, and late reports arrive. If you overwrite raw files, you lose the ability to reproduce an old training set.
- Keep fetchers per source and independent, so a BoM outage doesn't block the crash ingest.

Validate

Think in three layers, and decide for each check whether a failure **halts the run, quarantines the bad rows**, or only **warns**.

1. **Schema contracts:** expected columns, types and code domains. Check that severity codes, crash types and position types fall within known values. A new code appearing between releases is a signal to investigate, not to silently drop.
2. **Semantic and integrity checks:** these find the problems that matter for ML.
 - Every unit and casualty must reference an existing crash.
 - Crash-level severity should match the casualty table. A fatal crash needs at least one killed casualty.
 - Coordinates should fall inside SA and in the expected CRS. Older extracts state positional accuracy of about ± 100 m, which matters a lot later.
 - Report IDs should be unique, and timestamps should be plausible.
3. **Distributional checks across releases:** compare counts per year, the share in each severity class, and missing-value rates against the previous release.
 - The most recent year is almost always incomplete, because of reporting lag and severity still being finalised. Flag it as provisional. Don't let it train the model as if it were complete, or the model will learn that recent years are safer.
 - Property-damage-only crashes are systematically under-reported. You can't fix that in validation, but record it as a known bias.

Transform

Most of the intellectual work happens here.

Spatial assignment: is the crash at an intersection or on a segment?

- Snap each crash to the network. Treat it as an intersection crash if it falls within some radius of an intersection node, and as a segment crash otherwise. Cross-check against the dataset's own position or junction field.
- With ± 100 m positional error, a fixed radius will misassign crashes in dense CBD grids. Consider choosing the radius per context, and log the snap distance and how ambiguous the assignment was as a quality feature.
- **Segmentation is a modelling decision, not an ETL detail.** You can cut fixed-length segments, or homogeneous segments that break wherever attributes such as speed limit, AADT or cross-section change. Homogeneous segments give cleaner features. Very short segments create noisy zero counts, so set a minimum length.

Build two analysis tables:

- **Site-period panel**, for frequency and risk: one row per site per period, holding the crash count, AADT, segment length, and road attributes.
 - Counts are overdispersed and contain many zeros. That is why the safety-performance-function tradition uses negative binomial models with $\log(\text{AADT} \times \text{length} \times \text{time})$ as an offset.
 - Even if you use gradient boosting, keep exposure as an explicit offset or as features, so you are modelling rate rather than volume.
- **Crash-level table**, for severity: one row per crash, with an ordinal target (property damage only < minor < serious < fatal). The classes are heavily imbalanced, so plan for ordinal or cost-sensitive methods and avoid naive oversampling.

- **Combining them (“hybrid”):** multiply expected frequency by the severity distribution to get an expected harm score per site, such as EPDO or crash cost. Define the weights explicitly and version them.

Leakage is the biggest trap.

- For severity, only use features known *before* the crash outcome. Don’t use casualty count, hospitalisation, or units recorded post hoc if your use case is “predict severity given site, conditions and crash type”.
- Split data by **time**: train on early years, test on later ones. Group by **site**, so the same intersection never appears in both train and test. Random row splits will overstate performance badly.
- Keep **learned** transforms, such as target encoding, scaling and imputation fitted to the data, out of ETL. Those belong inside the model’s training pipeline, fitted on the training fold only. ETL should only do **deterministic** transforms: joins, spatial assignment, unit conversion, code mapping.

Other joins:

- **Weather:** nearest BoM station plus time window. Record the station distance, because rural SA stations can be very far apart.
- **Temporal features:** daylight versus dark (from sunrise and sunset, not the clock hour), day type, and school or public holiday.
- **Regression to the mean:** if the model will be used to rank black spots, sites selected for having had a bad year will improve on their own. That argues for Empirical Bayes-style shrinkage downstream, and for keeping multi-year history per site in the panel.

Load

- **Layered storage:** raw (immutable) → staged (validated, typed) → curated (the two analysis tables).
 - PostGIS suits the spatial work.
 - Partitioned Parquet suits the curated tables you train from.
- **Idempotent upserts** keyed on report ID plus release version, so rerunning a load never duplicates rows. Keep restated records as new versions rather than overwriting them, so you can answer “what did we know in 2025?”
- **Snapshot and lineage:** every training run should point to an exact curated snapshot, a code commit, and a validation report. Tools like DVC or MLflow dataset logging do this. Data versions matter more than code versions here, because the source changes underneath you.
- **Orchestration:** Airflow, Prefect or Dagster all work. Dagster’s asset model maps naturally onto “raw crash table → panel → severity table”. Make validation an explicit gate between staged and curated.
- **Monitoring after load:** record row counts, severity mix, snap-failure rate and provisional-year share per run, and check for drift between the newly loaded data and the training snapshot. This tells you when to retrain.

Suggested order to build in

1. Fetch and validate the crash tables alone.
2. Do spatial snapping, and check it by eye on a map of a few known intersections.
3. Build the severity table, which is simpler.
4. Add AADT to build the frequency panel. Expect exposure coverage to be the hardest gap, since many local roads lack counts.

5. Write the hybrid scoring last.

Sources:

- [Road Crash Data – data.gov.au \(DIT\)](#)
- [Road Crash Locations in SA](#)
- [SA Road Crash Statistics 2011–2015 \(AURIN\)](#)

You — Sat 26 Sept 20266, 14:22

the data would be sourced from data.gov.au, if we were to write code for fetching the API catalogue, can we put security measures in place to mitigate the tampering of the download files and or zip bombs that may be present in the dataset? would capping the file size limit help?

Claude — 14:22

Yes, you can and should add those protections. A file size cap helps, but only as one layer, and a naive cap won't stop a zip bomb. Below are the threats, what actually mitigates each one, and why.

Why a size cap alone isn't enough

A zip bomb is dangerous because of what it expands to, not because of its download size. Classic examples are a few kilobytes compressed and expand to gigabytes or petabytes. Capping the *download* stops oversized transfers but does nothing about expansion. You need limits at three points:

1. **Compressed bytes in.** Don't rely only on the Content-Length header, since it can be missing or wrong. Count bytes as you stream and abort when you pass the limit.
2. **Decompressed bytes out.** This is the cap that actually defeats bombs. The trap is that a zip's central directory *declares* each file's uncompressed size, and an attacker controls that number. Never trust it. Count the real bytes as they come out of the decompressor, in chunks, and stop the moment the running total crosses your ceiling.
3. **Compression ratio per entry.** Normal CSV compresses roughly 5–15×; bombs run thousands to millions to one. Abort if the bytes coming out exceed some multiple of the bytes going in, such as 100×. This catches a bomb early, before it reaches your absolute ceiling.

Rather than picking one arbitrary number, set limits **relative to history**: for example, abort if a release is more than 3–5× the size of the previous one. Crash data grows slowly, so a sudden jump is suspicious whatever its cause.

Other archive attacks worth covering

- **Nested archives.** A zip inside a zip inside a zip multiplies the expansion. Don't recursively extract. If an inner archive appears where you expected a CSV, treat that as a validation failure.
- **Overlapping-entry bombs.** These are non-recursive bombs where many directory entries point at the *same* compressed data. Defend against them by checking that each entry's byte range in the file doesn't overlap another's, and by capping the number of entries.
- **Zip slip (path traversal).** An entry named something like `../../etc/cron.d/x` writes outside your extraction folder. Normalise every entry path, reject absolute paths, `..` segments and symlinks, and confirm the resolved path stays inside the target directory.
- **Hidden archive formats.** `.xlsx`, `.docx` and KMZ are zip files internally, so they get the same limits. XML-based formats (KML, GML, XML metadata) can carry "billion laughs" entity expansion and XXE, an attack that reads local files. Parse them with a hardened parser (for example `defusedxml` in Python) and disable external entities.

- **HTTP-level compression.** If the server sends `Content-Encoding: gzip`, your HTTP client may decompress transparently. That is also a decompression bomb vector, so the decompressed-bytes cap must apply there too.
- **Malformed CSV.** Watch for extremely long lines, huge field counts, or embedded nulls. Cap line length and column count. Strip formula-leading characters (`=`, `+`, `-`, `@`) if outputs might ever be opened in Excel, since that enables CSV injection.

Tampering: what you can and can't defend against

It helps to separate *where* tampering could happen.

In transit: TLS largely solves this.

- Use HTTPS only, keep certificate verification on, and never disable it to “fix” an error.
- Keep a **host allowlist** and apply it to every redirect hop. This matters for `data.gov.au` specifically: it is largely a *harvester*, and CKAN resource URLs often point to the original publisher (`data.sa.gov.au`, DIT servers, or anywhere else). A resource URL is effectively untrusted input. Resolve it, check the final host against the allowlist, and refuse anything else, including private or internal IP ranges, so a poisoned URL can't make your fetcher probe your own network (SSRF).

At rest or at the source: This is harder, and you should be honest about the limit.

- CKAN metadata sometimes includes a `hash` or `size` field. These come through the same channel as the file, so they detect corruption but not a compromised publisher. If the publisher is compromised, the attacker controls both.
- **Cross-source comparison:** where the same resource exists on `data.sa.gov.au` and on `data.gov.au`, compare the hashes from both.
- **Record SHA-256 on every download** and store it with the raw file. You then have a tamper-evident history, and an unexpected hash change on a supposedly unchanged resource is an alert.
- **Treat your validation layer as tamper detection.** Poisoned data that parses cleanly still has to pass referential integrity, severity consistency and drift checks against previous releases. Subtle data poisoning, such as shifted severity labels, is far more likely to be caught there than at the network layer.

Contain the blast radius

Assume a check will eventually miss something. Then:

- Run fetch and extraction in an **isolated process or container** with memory, disk-quota and CPU limits. If a bomb gets past your logic, the operating system kills the process rather than your host filling its disk.
- Extract into a **temporary, size-limited directory**, and move files into the raw zone only after all checks pass.
- Set **connect, read and total wall-clock timeouts**, so a slow or stalled server can't hang the pipeline indefinitely.
- **Verify file types by magic bytes**, not by extension or the server's `Content-Type`. A `.csv` that starts with `PK\x03\x04` is actually a zip.
- **Never deserialise executable formats** (pickle, joblib) from external sources. Only accept inert formats like CSV, JSON and GeoJSON.
- **Least privilege:** the fetcher needs network egress and write access to a staging area only. It shouldn't hold database credentials or have write access to the curated layer.

Each of these limits is cheap. Together they turn “a hostile file crashed or poisoned the pipeline” into “a run failed with a clear reason and the previous good snapshot stayed in place”, which is the property you actually want.

You — Sat 26 Sept 2026, 14:23

so if we were to only use https, hash each file and limit the download size to 150MB (only if the dataset updates) would we mitigate most of our cybersecurity threats?

Claude — 14:23

No, not most of them. Those three measures cover the transport layer reasonably well, but they leave the threats specific to *processing* untrusted files wide open.

What your three measures cover

- **HTTPS (with certificate verification on)** stops interception and modification in transit. That’s a real threat handled.
- **Hashing each file** gives you a tamper-evident record and detects unexpected changes between runs. It **detects**; it doesn’t **prevent**. It only helps if you compare against something, such as the previous hash, the CKAN hash field, or the same file from another portal, and act on a mismatch. Without that comparison, a hash just records whatever you downloaded, poisoned or not.
- **A 150 MB download cap** stops oversized transfers and disk exhaustion from the download itself. It must be enforced by counting bytes while streaming, not by trusting Content-Length.

What they don’t cover

1. **Zip bombs:** this is the biggest gap. A 150 MB cap on the *compressed* file does nothing here. A 50 KB archive can expand to terabytes. You still need a cap on *decompressed* bytes, counted during extraction, plus a compression-ratio limit and no recursive extraction.
2. **Zip slip:** a malicious entry path like `../.. / somewhere` writes outside your extraction folder. Neither HTTPS nor hashing nor size limits touch this; only path validation does.
3. **Malicious redirects and SSRF:** HTTPS only proves you’re talking to *whichever* host you ended up at. CKAN resource URLs on data.gov.au often point to external publishers. Without a host allowlist on every redirect hop, the fetcher could be sent to an attacker’s HTTPS site, which has a perfectly valid certificate, or to internal addresses.
4. **Parser attacks:** XML entity expansion and XXE in KML/GML, CSV injection, and wrongly typed files (a “CSV” that’s actually a zip). These need hardened parsers and magic-byte checks.
5. **Source-side poisoning:** if the publisher’s copy is altered, HTTPS faithfully delivers the altered file and your hash faithfully records it. Only cross-source comparison and your validation and drift checks catch this.
6. **Hangs:** with no timeouts, a slow or stalled server can block the pipeline indefinitely, and a size cap never triggers.
7. **No containment:** if anything gets through, nothing limits the damage, because there’s no isolated process, no resource limits and no least privilege.

About “only if the dataset updates”

If that means applying the cap only on runs where the metadata says the dataset changed, be careful. Metadata comes through the same channel as the file, so it can be wrong or manipulated. The limits should apply to **every** download, without exception. Change detection decides *whether* to download, not *whether to check* what you downloaded.

A minimal set that covers most of it

Keep your three, and add these cheap ones:

- Host allowlist enforced on redirects
- Connect, read and total timeouts
- Decompressed-bytes cap and ratio cap during extraction, with no nested archives
- Path validation on archive entries
- Magic-byte type check and a hardened XML parser
- Hash compared against the previous release, with alerts on unexpected changes
- Extraction in a resource-limited container or process

With those, your coverage goes from “transport is safe” to “transport, archive handling and parsing are safe”. Source poisoning is then left to your validation layer, which is where it realistically gets caught anyway.

You — Sat 26 Sept 2026, 14:25

what if we only accept downloads from data.gov.au/DIT provided sites, rejects downloads over 150 MB, both from the declared size and while streaming, to prevent filling the disk?

Claude — 14:25

That’s a real improvement. Those two controls together close the redirect/SSRF gap and the download disk-exhaustion gap. But the biggest remaining risk is the zip bomb, and a download cap still doesn’t touch it.

What this now covers

- **Malicious or unexpected hosts:** the allowlist stops the fetcher being steered to an attacker’s site or to internal addresses, provided it is implemented carefully (below).
- **Disk exhaustion from the download:** checking the declared size rejects oversized files early, and counting bytes while streaming catches a server that lies about or omits Content-Length. Together that is the correct approach.

Getting the allowlist right

Allowlists often fail in the details:

- **Match exact hostnames after parsing the URL.** Never use substring or “starts with” checks. Those let through `data.gov.au.evil.com`, `evil.com/data.gov.au` and `https://data.gov.au@evil.com`; in the last one, everything before @ is just a username.
- **Check every redirect hop**, not just the first URL. Better still, turn off automatic redirects and validate each Location header yourself.
- **Find out where the files actually live.** `data.gov.au` and `data.sa.gov.au` are separate hosts. CKAN download links often point to cloud storage or a DIT server rather than the portal itself. Inspect the real resource URLs and allowlist those exact hosts. Otherwise you’ll either block legitimate downloads or be tempted to loosen the rule.
- **Block private and internal IPs after DNS resolution.** This stops an allowlisted name that resolves to something unexpected (DNS rebinding).
- **Keep TLS certificate verification** on, so an allowlisted name actually proves who you’re talking to.

What's still uncovered

1. **Zip bombs: the main gap.** Your cap protects the disk from the *download*. Extraction is a separate step where a file under 150 MB can expand without limit. You need a cap on *decompressed* bytes, counted as they come out of the decompressor rather than read from the archive's declared sizes, plus a ratio limit, an entry-count limit and no nested extraction.
2. **Zip slip:** validate entry paths so extraction can't write outside its folder.
3. **Parser attacks:** use a hardened XML parser, magic-byte type checks, and CSV line and column limits.
4. **Timeouts:** a server that trickles bytes slowly never reaches 150 MB but can hang the pipeline indefinitely. Set connect, read and total timeouts.
5. **Source-side tampering:** the allowlist guarantees the file came *from* DIT or data.gov.au, not that the content is genuine. Compare hashes against the previous release, and rely on validation and drift checks.
6. **Containment:** extract in a resource-limited process or container, so anything that slips through can't take down the host.

Short answer

The allowlist plus the dual size check handles **where** the data comes from and **how big the download is**. To mitigate most threats you also need to handle **what happens when you open it**. At minimum, add:

- a decompression cap
- path validation
- timeouts
- hash comparison against the previous release

You — Sat 26 Sept 2026, 14:26

we would also need to have something in place to check that the file is a zip and not a html as well as something to refuses entries whose compression ratio or unpacked size is strange (a tiny file that expands to gigabytes)

Claude — 14:26

Yes, both are worth adding, and they cover two of the gaps from before. Here's how to make each one effective rather than easy to get around.

Checking it's actually a zip, not HTML

This matters in practice. Portals and CDNs often return an HTML error page, login page or bot-check page with a **200 OK** status. Without this check, your pipeline will quietly "download" a web page and fail confusingly later, or worse, pass it along.

- **Check the magic bytes, not the extension or Content-Type.**
 - Both the extension and Content-Type are just labels the server chose.
 - A local zip file entry starts with PK\x03\x04.
 - An *empty* zip starts with PK\x05\x06. Reject that too, since an empty release is an error, not a success.
 - HTML usually starts with <!DOCTYPE or <html, sometimes after whitespace or a byte-order mark.

- **Also check the HTTP status** and reject anything other than 200. Then still do the byte check, since the 200-status error page is the common failure.
- **Confirm the archive parses, not just its first bytes.** Zip readers locate the archive from its *end* (the central directory), not its start. A file can therefore begin with something else and still open as a zip, or begin with PK and be truncated. The real test is: the start bytes look like a zip, *and* the central directory parses, *and* every entry is consistent with it.
- **Check the entries are what you expect.** The archive should contain CSVs with expected names, not HTML, executables or nested zips. Apply the same magic-byte idea to each extracted file.

Refusing entries with suspicious ratios or sizes

Use two layers. The first is cheap but can be lied to; the second can't.

1. Pre-check using declared sizes (before extracting anything)

- Read each entry's declared compressed and uncompressed size from the central directory.
- Reject the archive if any entry's declared ratio, the total declared size, or the entry count exceeds your limits.
- This catches lazy bombs instantly. But remember the attacker writes these numbers, so this is a filter, not the actual protection.

2. Enforcement while extracting (the real control)

- Decompress in chunks and keep a running count of actual bytes produced, per entry and in total.
- Abort the moment either:
 - **actual output exceeds the declared size** for that entry, which means the archive is lying, so reject the whole thing; or
 - **the running output-to-input ratio** or absolute size crosses your thresholds.
- Clean up partial output on abort.

Calibrating the thresholds so they don't misfire

- **Measure real releases first.** Crash CSVs are repetitive (codes, suburb names, blanks), so ratios of 10–30× are normal and some may reach 50–100×. Set the ratio limit comfortably above what genuine files show, such as a few hundred×, and well below bomb territory (thousands to millions).
- **Apply the ratio check only above a minimum size.** A 2 KB file that expands to 200 KB has a 100× ratio and is harmless. Without a floor, small files cause false positives.
- **Keep an absolute ceiling as well.** A large, genuinely compressible file can have a normal ratio and still unpack to something enormous. The ratio catches "tiny file, huge output"; the ceiling catches "big output, full stop". Base the ceiling on history, such as a multiple of the last release's unpacked size.

Related checks that belong in the same step

- Allow only standard compression methods (stored and deflate). Reject unusual ones, which you would never expect from DIT.
- Reject encrypted entries.
- Reject nested archives instead of extracting them.
- Cap the entry count.
- Reject entries whose byte ranges overlap. That trick is how non-recursive zip bombs work.

Where you now stand

Combined with your earlier controls (HTTPS, exact-host allowlist, dual size cap, hashing), this covers transport, source host, download size, file type and decompression. The items still missing are:

- **path validation** on entry names (zip slip)
- **timeouts**
- **hash comparison against the previous release**, not just recording the hash
- **running extraction in a resource-limited process**, as a backstop

All four are small additions.

You — Sat 26 Sept 2026, 14:28

yeah so we'd need to check the hash of each file and compare it to the previous release, making sure that if it changes then we completely stop running the system, can we move onto the data cleaning process now?

Claude — 14:28

One correction before moving on: halting whenever the hash changes would stop the system on every legitimate release, because a new release *is* a changed file. The hash comparison means something different depending on what the metadata says:

Metadata says	Hash	Meaning	Action
Not updated	Unchanged	Nothing new	Skip quietly
Updated	Changed	Normal new release	Continue to validation
Not updated	Changed	File changed silently	Halt and alert: possible tampering or an unannounced restatement
Updated	Unchanged	Metadata touched but data identical	Skip, log it

So the halt condition is an **unexplained** change, not any change. For legitimate changes, the validation and drift checks act as the second gate.

Data cleaning

The aim is to turn the raw tables into consistent, typed, trustworthy data **without making modelling decisions**. Cleaning should be deterministic and reversible. Flag problems instead of deleting rows, and log every rule you apply. Anything learned from the data (imputation values, encodings, scaling) belongs in the model pipeline, fitted on training data only.

The field names below are from memory of the SA crash release format. Check them against the current data dictionary before relying on them.

1. Harmonise across releases

- **Column names and order drift** between years. Keep a mapping table from every historical name variant to one canonical name, and fail loudly on an unmapped column instead of silently dropping it.

- **Coded fields often arrive as combined strings**, such as "3: SI" for severity. Split these into a numeric code and a label, and validate both against a reference list. Labels sometimes change wording between years while codes stay stable, so key on codes.
- Normalise whitespace, case and encoding. Suburb and LGA names in particular need a canonical lookup, because councils have merged and been renamed.

2. Treat "unknown" as information, not as a blank

There are several kinds of missing value, and they mean different things:

- **True blanks**: the field wasn't filled in.
- **Explicit unknowns**: values like "Unknown" or "Not Stated".
- **Not applicable**: for example, helmet use for a car occupant.
- **Sentinel values posing as data**: an age of 0, a vehicle year of 0 or 9999, a speed zone of 0.

Map all of these to explicit categories rather than to a single null. Missingness here is rarely random: minor crashes typically have more fields recorded as unknown than fatal ones, which are investigated thoroughly. If you impute or drop missing values during cleaning, you'll lose a signal that correlates with severity, or worse, create one artificially.

3. Rebuild time properly

- Releases tend to split date and time into separate fields (year, month, day of week, time of day) rather than giving a timestamp. Rebuild a proper local timestamp where you can, and record when you can't.
- Watch for **heaping**: reported times cluster at round hours and half-hours. Treat time as approximate, and prefer coarser features such as hour band or daylight versus dark.
- Pin the timezone to Australia/Adelaide, and handle daylight-saving transitions explicitly.
- If the release includes a day/night field, cross-check it against computed sunrise and sunset. Disagreements tell you about recording quality.

4. Clean the coordinates

- Confirm the coordinate system. SA data has historically used MGA zone 54 (GDA94). Reproject everything to one system, ideally GDA2020, and record the transformation.
- Flag null, zero or out-of-state coordinates, and flag points that fall in the sea or far from any road.
- Keep any location-grouping identifier the release provides, and keep the positional-accuracy caveat attached to the record. Both feed into the intersection-versus-segment snapping later.

5. Handle duplicates and revisions

- Report IDs should be unique within a release. Across releases, the same crash may reappear with revised values, for example severity upgraded after a death within 30 days. Use the newest version, keep earlier versions in history, and record which release each value came from.

6. Check consistency between tables

- Crash-level totals (units, casualties, fatalities, serious injuries) should match the row counts in the units and casualty tables.
- Crash severity should match the worst injury in the casualty table.
- Mismatches are where data-entry errors and revisions show up. Flag and count them rather than "fixing" one side, unless you know which table DIT treats as authoritative.

7. Check range and plausibility

- Speed zones should come from the legal set of values.
- Driver age should be in a plausible range, and licence type should be consistent with age.
- A vehicle's year should not be later than the crash year.
- Occupant counts should be at least the number of casualties in that unit.
- Flag violations as quality issues. Don't clip or alter values in cleaning.

8. Tag each column by when it's known (leakage prep)

Add metadata to every column saying when its value becomes known:

- **Before the crash:** road and site attributes, speed zone, traffic controls, alignment.
- **At the time of the crash:** weather, light, surface condition, crash type, and arguably vehicles and drivers involved.
- **After the crash:** casualty totals, injury extent, hospitalisation, responsible-unit assessment, and often DUI or drug findings, which depend on later testing.

The cleaning stage doesn't decide which of these the model uses. It makes the decision explicit and machine-checkable, so a post-crash field can't slip into a severity model by accident.

9. Mark known biases rather than correcting them

- Flag the most recent year as **provisional**.
- Note that property-damage-only crashes are under-reported, and that reporting practice may have changed over time. If you find a step change in PDO counts at a particular year, record it as a likely reporting-regime change, not a real-world safety improvement.

10. Privacy

The units and casualty tables contain age, sex and residential postcode. That is low-risk on its own, but when combined with precise location and time it becomes more identifying. Decide early whether these fields are needed, coarsen them if they are (age bands, region instead of postcode), and keep the fine-grained versions out of anything published, including dashboards.

Outputs of this stage

- Cleaned, typed tables keyed on report ID and release.
- A per-row **quality-flags** column, with no rows silently removed.
- A **cleaning report** for each run: counts per rule fired, number of unknowns per field, consistency-mismatch rates, and coordinates rejected.

Compare that report run to run as well. A sudden jump in any rule's firing rate is often the first sign of an upstream format change.

You — Sat 26 Sept 2026, 14:28

is there a python library that can check types, ranges and uniqueness for every dataset?

Claude — 14:29

Yes, there are several. For a pipeline like yours, **Pandera** is the best fit, but here are the main options and when each makes sense.

Pandera (recommended)

You declare one schema per table, listing each column's type, allowed range or set of values, whether nulls are allowed, and whether it must be unique. You then validate each DataFrame against that schema.

- **Covers all three requirements directly:**
 - **Types:** with optional coercion.
 - **Ranges:** greater-than/less-than checks, or membership in an allowed set, e.g. legal speed zones or severity codes.
 - **Uniqueness:** single columns, or column combinations such as report ID plus release.
- **Lazy validation** collects *every* failure in one pass, returned as a table of which rule failed, on which column and row. That table can become your cleaning report directly instead of stopping at the first error.
- **Custom checks** handle cross-column rules, such as vehicle year not after crash year, or occupants at least the casualty count.
- **Schema inference** generates a draft schema from an existing dataset, which you then tighten by hand. This saves a lot of typing for wide tables.
- Works with pandas and also Polars, which is worth considering for larger multi-year files.
- Schemas are plain Python, so they live in version control next to your column-mapping table.

Great Expectations (GX)

- More of a framework than a library, with suites of “expectations”, stored validation results and generated HTML data docs.
- **Strengths:** good for documenting and auditing data quality across many datasets and teams, and for profiling.
- **Weaknesses:** much heavier setup, and its API has changed significantly between major versions, so older tutorials often don't match the current release.
- Choose it if stakeholders need browsable quality reports; it's overkill for a single pipeline.

Frictionless

- Validates files against **Table Schema**, a JSON standard from the open-data world. Some CKAN portals already publish or support these schemas.
- Validates the *file itself* before you load it into a DataFrame: encoding, header row, blank rows, row shape, types, constraints, uniqueness.
- Fits nicely as the first check right after extraction, before Pandera checks the cleaned tables.

Others worth knowing

- **Pydantic:** validates record by record. Excellent for API responses, such as checking the CKAN metadata JSON you fetch. Slow and awkward for million-row tables.
- **Soda Core:** checks written in YAML, aimed mainly at data already in a warehouse or database, e.g. post-load checks in PostGIS.

How these could fit your pipeline

1. **Pydantic:** validate the CKAN API metadata before trusting any URL or timestamp in it.
2. **Frictionless (optional):** check raw file structure immediately after extraction.
3. **Pandera:** one schema per table (crash, units, casualty), run on the harmonised data. Use lazy mode so the failure table feeds your quality flags and cleaning report.

4. **Custom code:** the cross-table checks, such as crash totals matching the casualty rows and severity matching the worst injury, still need a bit of your own logic. None of these libraries handles relationships across tables well out of the box. You can still express them as Pandera checks on a joined or aggregated frame, so all failures end up in one report format.

One design tip: keep the allowed value sets (severity codes, speed zones, crash types) in a separate reference file that the schemas read from. When DIT adds or renames a code, you then update a lookup instead of editing validation code. An unknown code then fails validation clearly instead of being silently accepted.

You — Sat 26 Sept 2026, 14:29

can we use pandera to move bad rows to a different file with the rule they broke instead of just removing them?

Claude — 14:30

Yes. Pandera doesn't have a built-in "send bad rows to a file" option, but its lazy validation gives you exactly what you need to build one.

How it works

1. **Validate in lazy mode.** Pandera then doesn't stop at the first problem. It raises one exception holding a **failure-cases table** with one row per failure. Columns include:
 - which schema and column failed
 - which check failed, e.g. `in_range(0, 110)` or `field_uniqueness`
 - the offending value
 - the **index** of the row it came from
2. **Use that index to split the data.** Rows whose index appears in the failure table go to quarantine; the rest continue as clean data.
3. **Collapse multiple failures per row.** One row can break several rules, so group the failure table by index and combine the rule names. Each quarantined row then carries a list of everything it violated, not just one reason.
4. **Write the quarantine file:** the original row in full, the rules broken, the failing values, plus run ID, release version and timestamp. Parquet or CSV works; a separate table in PostGIS also works if you want to query it.

Pandera also has a `drop_invalid_rows` option, but as the name says it just **drops** them. That's exactly what you're trying to avoid, so use the failure-cases approach instead.

Things to get right

- **Not every failure belongs to a row.** If a whole column is missing, or a column can't be converted to its type at all, the failure-cases entry has **no row index**. That's a file-level problem, likely a format change by DIT, and should **halt the run**. Quarantining it would mean quarantining the entire file. So route failures in two directions: index present → quarantine the row; index empty → stop.
- **Use a stable index.** Set the DataFrame index to the report ID (plus release) before validating. A positional index is fragile: it shifts if you filter or concatenate, and it means nothing to someone reading the quarantine file later.
- **Uniqueness failures need a policy.** For duplicate report IDs, Pandera can report *all* copies, or every copy except the first or last. Usually you want all copies quarantined together, so someone can see the conflict instead of having the pipeline silently pick one.

- **Separate “reject” from “flag”.** Some rules mean the row is unusable, such as a missing report ID or an invalid severity code. Others mean the row is suspicious but usable, such as time recorded at exactly midnight, or a coordinate far from a road. Run two schemas: a strict one whose failures are quarantined, and a soft one whose failures only add quality flags to rows that stay in the clean data. (Pandera’s warning-only checks don’t show up in the failure table, so a separate soft schema works more reliably.)
- **Put a ceiling on quarantine.** If more than a set share of rows are quarantined, say 2–5%, judged against your historical rate, treat it as a pipeline failure and halt. A sudden surge almost always means something changed upstream, not that thousands of crashes are individually bad.
- **Cross-table checks work the same way.** Run them on a joined or aggregated frame indexed by report ID, so their failures land in the same quarantine format.

What you end up with

- **Clean data:** passed the strict schema, with soft-schema flags attached.
- **Quarantine file:** every rejected row in full, with all the rules it broke, traceable to a run and release.
- **Run summary:** counts per rule. Compare this between runs to spot upstream changes.

Because quarantined rows are kept rather than deleted, you can also review them later and add a corrected version through the normal pipeline. The fix is then recorded rather than being a manual edit somewhere.

You — Sat 26 Sept 2026, 14:33

[attached file: SchoolCrossings_GDA2020.geojson] [attached file: TrafficSignals_GDA2020.geojson] [attached file: PedestrianCrossings_GDA2020.geojson] [attached file: TrafficVolumeEstimates_WGS1984.geojson] [attached file: RoadCrashes_GDA2020.geojson] these are the datasets we were using, again were dealing with just concepts so don't write any code yet. Is there a way we could address the different types of measurements used in the datasets? like a universal measurement of meters?

Claude — 14:33

I'll look at how each file stores coordinates and distances before answering.

Actions taken

- Ran a command: Preview the start of each GeoJSON file
- Ran a command: Check CRS declarations, geometry types and coordinate ranges

Yes. Standardise everything to metres by projecting all five datasets into one metric coordinate system. Before deciding which, it helps to know what the files actually contain, because there are more kinds of measurement than the file names suggest.

What I found in the files

All five store coordinates as **longitude/latitude in decimal degrees**, not metres. That includes the four labelled GDA2020. The labels only differ in the datum (the model of the Earth’s shape behind the coordinates).

Dataset	Geometry	Measurements inside
Road crashes	43,522 points	Degrees. Counts per location for 2020–2024 combined

Dataset	Geometry	Measurements inside
Traffic volume estimates	2,685 lines	Degrees. SHAPE_Length in degrees . Start/end road reference distance in km . Volume in vehicles/day , with a 2018 base year
Traffic signals	1,552 points	Degrees. Road reference fields exist but were empty in the sample
Pedestrian crossings	642 points	Degrees. Distances in free text, e.g. “21m N of Seafield Avenue”
School crossings	236 points	Degrees. Same kind of free-text distances (“Approx. 130m East of...”)

None of the files declares its coordinate system internally. The only clue is the file name.

Why degrees can't be used for distance

A degree isn't a fixed length:

- One degree of latitude is always about 111 km.
- One degree of longitude shrinks as you move away from the equator: about **100 km** at the top of SA, **91 km** in Adelaide and **88 km** at Mount Gambier.

So a “50 m” buffer written in degrees is a stretched oval whose size changes across the state. The SHAPE_Length field shows the problem: its median is 0.009 “degrees”, which means nothing as a road length.

The approach: one projected coordinate system in metres

1. Choose a statewide system. A projected coordinate system flattens the Earth onto a plane where the units are metres.

- **GDA2020 / SA Lambert (EPSG:8059)** is the best fit. It is designed to cover all of South Australia with low distortion, so distances and buffers behave consistently from the far west to the South East. It's the usual choice for statewide SA analysis.
- **GDA2020 / MGA** is common, but SA spans three MGA zones (52, 53 and 54). Forcing everything into one zone distorts the edges, and splitting by zone makes cross-zone joins awkward.
- **GDA2020 / Australian Albers (EPSG:9473)** preserves area. It's better if you ever need densities per km², but slightly worse for distances.

2. Reconcile the datums before projecting. One file is WGS84 and the others are GDA2020.

- The gap between the two is currently under a metre, because GDA2020 is fixed to the Australian plate and WGS84 drifts with it by about 7 cm a year.
- Compared with the crash data's roughly ±100 m location accuracy, that gap doesn't matter in practice. Still, make the decision explicitly: either transform properly or record “treated WGS84 as GDA2020, error under 1 m” in your pipeline config. Don't let it happen silently.
- Because the files don't declare a coordinate system, record the expected one **per source in config**. Then validate it with a bounding-box check: values around 129–141 and –26 to –38 confirm degrees in longitude/latitude order. If coordinates arrive in the hundreds of thousands, they're already projected in metres and the check should fail loudly. This fits into the validation stage from earlier.

3. Keep both versions. Store the projected metre geometry for analysis (snapping, buffers, nearest-signal distances). Keep the original longitude/latitude for web maps and display, since web maps expect degrees.

The other measurements

- **SHAPE_Length:** discard it and recompute each segment's length in metres after projecting.
 - Use the road reference distances (end minus start, $\text{km} \times 1,000$) as a **cross-check**. The two lengths should roughly agree. A large mismatch means a broken or mismatched geometry, and becomes a validation flag.
- **Road reference distances (km):** convert to metres, but treat them as a second, one-dimensional coordinate system: distance *along* a numbered road.
 - This is useful for splitting roads into segments, and for placing an asset such as a signal on a road by road number and distance. That's often more reliable than snapping by straight-line distance.
- **Free-text distances** ("21m N of..."): treat them as descriptions only. The point geometry already holds the location, and parsing text to rebuild positions adds error.
- **Too many decimal places:** coordinates are stored to 15 decimal places, implying sub-nanometre precision. Six decimal places is about 0.1 m. Store what you receive, but don't let it imply accuracy the crash data doesn't have.

Units beyond distance

Distance isn't the only unit you need to standardise. Your risk measure combines several:

- Crashes are **5-year totals** (2020–2024).
- Volumes are **vehicles per day**, estimated for **2018**.
- Lengths will be in **metres**.

To compare sites fairly, turn these into one exposure unit, typically **vehicle-kilometres travelled**: daily volume $\times$ length in km $\times$ 365 $\times$ number of years. Risk then becomes crashes per million vehicle-km for segments, or per million entering vehicles for intersections. The 2018 volumes against 2020–2024 crashes are a time mismatch, with COVID-era traffic in the middle of it. Note it as a known limitation rather than ignoring it.

A convention that makes this stick

Put the unit in every column name: `length_m`, `dist_to_signal_m`, `aadt_veh_per_day`, `exposure_mvkt`. Your Pandera schemas can then enforce a plausible range for each unit. For example, a segment `length_m` between 1 and 50,000, or a nearest-signal distance that can't be negative. A degrees-versus-metres mix-up then fails validation instead of quietly producing wrong features.

One more thing the files showed: the crash dataset is **already aggregated per location**. Each point holds counts by severity and crash type across 2020–2024, not individual crashes. That changes what I said earlier about a per-crash severity table. With this dataset, severity modelling works at the location level, such as the share of serious and fatal crashes at a site. Individual crash records, with per-crash weather and time, would need DIT's unit-level crash data.

You — Sun 27 Sept 2026, 14:36

so would using a EPSG:7845 work here?

Claude — 14:36

Actions taken

- Ran a command: Compare scale distortion of EPSG:7845 and 8059 across SA
- Ran a command: Install pyproj and compare projection distortion

Yes, EPSG:7845 would work, but EPSG:8059 (GDA2020 / SA Lambert) is the better fit for a project limited to South Australia.

EPSG:7845 is **GDA2020 / GA Lambert**, Geoscience Australia's national projection. It uses the same datum as four of your five files and measures in metres, so it meets the basic requirement. The difference is how much it distorts distance across SA.

Distortion compared

I worked out the scale error each projection introduces at points across the state:

Location	EPSG:7845 (national)	EPSG:8059 (SA Lambert)
Far north (−26°)	−1.21%	+0.30%
West coast	−1.00%	−0.23%
Eyre Peninsula	−0.78%	−0.24%
Adelaide	−0.28%	−0.11%
Mount Gambier	+0.58%	+0.28%

The cause is in how each projection is set up:

- **EPSG:7845** is tuned for the whole continent. Its two lines of true scale sit at 18°S and 36°S, so northern SA falls in the middle, where it understates distances the most.
- **EPSG:8059** puts its lines of true scale at 28°S and 36°S, which bracket SA closely. As a result, its error stays within about ±0.3% everywhere in the state.

Does the difference matter in practice?

- **Buffers and snapping:** barely. A 1.2% error on a 50 m buffer is 0.6 m, which is tiny next to the crash data's ±100 m location accuracy.
- **Segment lengths and exposure:** this is where it shows. Rural segments in the north and west would come out about 1% too short, so vehicle-km would be slightly too low and crash rates slightly too high. The bias also runs consistently by region: it is larger in the north and smaller in Adelaide. In a model, a regional bias is worse than random noise, because the model can learn it as if it were real.

You could avoid this by calculating lengths on the ellipsoid (geodesic lengths) instead of on the flat projection. However, using a projection that suits SA means you don't have to think about it at all.

A practical detail if you did use 7845

EPSG:7845's false origin is at 0°, 134°E. Every northing is therefore negative, and every easting west of 134°E (most of western SA) is negative too. That isn't wrong, but it can break range checks or other code that assumes projected coordinates are positive.

SA Lambert adds offsets (1,000,000 m east and 2,000,000 m north), so all its coordinates in SA are positive. In your Pandera schema, that gives you a simple check that catches data still in degrees or in the wrong system: eastings and northings must fall within SA Lambert's expected range.

Recommendation: use EPSG:8059 as the canonical coordinate system for analysis. EPSG:7845 would only be worth choosing if the project later grew to cover other states.

You — Sun 27 Sept 2026, 14:38

thats fine, we would then need to merge duplicate columns and stuff of the same points

Claude — 14:38

I'll check the files for what "duplicate" actually looks like before suggesting rules.

Actions taken

- Ran a command: Profile duplicate IDs, coordinates and overlaps in each dataset
- Ran a command: Measure spread of duplicate-ID points and crash count inconsistencies
- Ran a command: Check crash column relationships and cluster duplicate-ID sites

I checked all five files. Your "duplicates" are actually four different problems, and each needs its own rule. Merging them with one blanket approach would introduce errors.

1. Crash columns that overlap

These look like separate severity columns you could add up, but some of them overlap:

- `cse_pdo + cse_inj + cse_fat` equals `total_crashes` at all 43,522 locations.
- `cse_si` (serious injury) is **a subset of** `cse_inj`, not a separate category.
- So adding all four severity columns double-counts at 3,178 locations.

Don't merge these columns. Turn them into categories that don't overlap: property damage only, minor injury (`cse_inj - cse_si`), serious injury, and fatal. That's also the order your severity target needs.

Two other things to handle in this table:

- `total_casualties`, `total_fatalities` and `total_serious_injuries` count **people**, while the `cse_*` columns count **crashes**. Keep them apart.
- `cty_hit_parked_vehile` is misspelt. Fix it in your column-mapping table.

Turn the relationships into Pandera checks:

- `PDO + injury + fatal = total`
- `serious ≤ injury`
- crash types sum to the total (this holds now)
- `night crashes ≤ total`

2. Several points for one physical site

Signals, pedestrian crossings and school crossings often have several points with the same ID a few metres apart. For example, the Fullarton Rd / Greenhill Rd signals appear as four points about 15 m apart, probably one per corner or approach. The median spread within an ID is 12–20 m.

Merge these into one site:

- Geometry: the centroid of the points.
- Description: keep the most common one, and log the variants.
- Source IDs: keep a list of all the original records, for lineage.
- Point count: keep it as a feature, because more signal points usually means a larger or more complex intersection.

3. One ID used for different places

This is the dangerous one. The same signal number appears at places up to **396 km apart**, and pedestrian crossing numbers up to 52 km apart. Some IDs show up on different streets, e.g. one ID labelled both “Fullarton Rd – Greenhill Rd” and “Grand Junction Rd; Addison Rd”. So `ts_no` and `site_no` are **not unique keys**.

Dataset	Records	Distinct IDs	Distinct sites (grouped by ID + distance)
Traffic signals	1,552	718	~901
Pedestrian crossings	642	428	~457
School crossings	236	225	225

Rule: group by ID *and* distance. Points with the same ID that sit close together form one site; the same ID far away is a different site. I used 60 m for the counts above, but set your threshold from the actual distribution. It splits cleanly: same-site distances are tens of metres, different-site distances are kilometres. Let distance decide, and use the descriptions as supporting evidence. Where they disagree (close together but different descriptions, or far apart with the same description), send the group to quarantine for review instead of guessing.

Then give each site its own key, such as `asset_type` + `sequence`, and keep a mapping table from site to original records. Version that mapping table so the merge can be reproduced.

Don’t merge across datasets. A pedestrian crossing inside a signalised intersection is still useful information. Keep the three asset types separate, and when you build site features, record each one’s presence and distance, e.g. `has_signal_within_50m` or `dist_to_school_crossing_m`.

4. Crash locations that are almost the same place

`unique_loc` isn’t an arbitrary ID: it is the location itself. It’s a 7-digit SA Lambert easting followed by a 7-digit northing. Decoded, it lands within about 3 m of my GDA2020 SA Lambert projection, which is consistent with it having been calculated in the older GDA94 datum. Three consequences:

- IDs that differ by one digit are 1 m apart. There are 3 pairs at identical coordinates and about 2,800 pairs within 5 m.
- You don’t need to merge these at the point level. They combine naturally when you snap crashes to intersections and segments.
- When you do combine them, **sum** them. Every column is a count, so summing is valid. Never average counts, and never sum rates.
- You get a free validation check: decode the ID and compare it with the geometry. A gap of more than a few metres means the geometry has been corrupted.

5. Traffic volumes

There are no duplicate segment IDs and no overlapping road + chainage ranges, so there’s nothing to merge. Two things to settle, though:

- **Carriageway codes:** U (1,885 segments) and L (800). U is probably undivided and L one side of a divided road, but there’s no matching R. Check the data dictionary to find out whether L volumes cover one direction or both before using them as exposure. Getting this wrong halves or doubles exposure on every divided road.
- **Base years range from 2017 to 2024**, not only 2018 as I said earlier. Keep the year on each segment, and treat older estimates as less reliable.

General merge rules

- **Counts:** sum.
- **Categorical attributes:** they must agree, otherwise flag the conflict.
- **IDs:** keep all of them as a list.
- **Geometry:** use the centroid.
- **Descriptions:** keep the most common one, and log the rest.

Order of operations: project to EPSG:8059, fix the crash columns, merge duplicates within each dataset, then snap everything to the road network. After merging, check with Pandera that site keys are unique, and flag any two sites of the same type closer than your threshold, since that means the merge missed something.

You — Sun 27 Sept 2026, 14:40

yeah that makes sense

Claude — 14:40

Good. The next stage would be snapping crashes to intersections and road segments, which is where most of those merge rules come into play.

There are two things to decide there:

- **Intersection radius:** how close a crash has to be to an intersection before it counts as an intersection crash.
- **Segment boundaries:** how to split roads into segments, using the traffic-volume lines and their road reference distances.

I can go through either of those when you're ready. Before then, it's worth checking the data dictionary for the U/L carriageway codes, because that answer changes the exposure calculation.

You — Sun 27 Sept 2026, 14:40

20 meters

Claude — 14:40

I'll check how 20 m fits the actual crash positions around the signalised intersections.

Actions taken

- Ran a command: Measure how crashes are distributed by distance from signal sites

Yes, 20 m is a good choice, and your data supports it. I measured crashes by distance from the 901 signalised intersection sites (after merging their duplicate signal points). Within 200 m of a signal there are 23,440 crashes. This is how densely they cluster, ring by ring:

Distance from signal centre	Crashes per m ² (relative)	Cumulative share
0–5 m	41.8	14%
5–10 m	20.7	35%
10–15 m	5.3	44%
15–20 m	1.2	47%

Distance from signal centre	Crashes per m ² (relative)	Cumulative share
20–30 m	0.4–0.6	50%
50–75 m	0.20	65%
150–200 m	0.05	100%

Density falls about 35-fold between the centre and 20 m, then levels off. That bend is the natural edge of “crashes at the intersection”, and it falls right at your 20 m. The sharp peak in the first 10 m probably also reflects DIT recording intersection crashes at the intersection point itself, which makes the boundary clean.

The table also shows the limits of a single radius:

- **Approach crashes.** Between 20 m and about 75–100 m, density is still 3–4 times the background level at 150–200 m. These are mostly rear-ends in queues and crashes at turn lanes. They are caused by the intersection but happen on the approach. A strict 20 m rule assigns them to the road segment, which inflates segment risk next to busy intersections. A sensible fix is two tiers:
 - **0–20 m:** intersection crash.
 - **20–~75 m:** intersection-*related*. Assign these to the segment but flag them, so you can test both ways of counting them.
 - I believe the US Highway Safety Manual uses about 76 m (250 ft) as its intersection-related distance. That matches where your density curve flattens.
- **Large intersections.** I measured from each site’s centre point. Big multi-lane intersections, with up to 7 signal points, extend further than small ones. You could scale the radius per site, for example 20 m plus the spread of its signal points.
- **Intersections close together.** Where two intersections are less than 40 m apart, their zones overlap. Assign each crash to the nearest one and flag it.
- **Unsignalised intersections.** This analysis only covers signals. Roundabouts, stop and give-way junctions need the nodes of a road network layer, which none of the five files provide.

Because this decision affects every crash count, run a quick sensitivity check: build the site table at 15, 20 and 30 m. If the model’s rankings barely change, 20 m is safe. If they shift a lot, report that alongside your results.

You — Sun 27 Sept 2026, 14:41

we need something that can write three csv tables and a data contract that the ML module validates against

Claude — 14:42

Here’s how I’d structure that. The three tables follow from the decisions so far. I’ve assumed they are an intersections table, a segments table and a crash-assignment table; tell me if you meant a different three.

The three tables

1. intersections.csv: one row per intersection site.

- `intersection_id`: your own key, not the reused `ts_no`
- Location: `x_m` and `y_m` in EPSG:8059
- Asset features: signal-point count, presence of and distance to pedestrian and school crossings

- Approach volumes, where you can link them
- Crash counts by severity (property damage only, minor, serious, fatal), using the 0–20 m rule
- Data window, e.g. 2020–2024
- Source record IDs, for lineage

2. segments.csv: one row per road segment.

- `segment_id`
- Road number, start and end road reference distance, carriageway code
- `length_m`, recomputed after projection
- `aadt_veh_per_day` and its base year
- `exposure_mvkt` (exposure in million vehicle-km travelled)
- Crash counts by severity
- A separate count of intersection-related crashes (those 20–75 m from an intersection)

3. crash_locations.csv: one row per crash location, linking it to the site it was assigned to.

- `unique_loc`
- `site_type` (intersection or segment) and `site_id`
- `snap_distance_m`
- Assignment flags: ambiguous, overlapping zones, or intersection-related
- The original count columns

This table makes the other two auditable. Summing its counts by site must reproduce the counts in tables 1 and 2 exactly. It also lets you rebuild sites at 15 m or 30 m for the sensitivity test without rerunning the whole pipeline.

Quarantined rows stay out of these three tables and go to their own file. The ML module should never read them.

The data contract

The contract is one versioned file that the pipeline writes against and the ML module validates against. Both sides read the same file, so it is the single source of truth and neither side keeps its own copy of the rules.

What the contract specifies:

- **Per column:**
 - name and type
 - unit, in both the column name and the metadata
 - whether nulls are allowed, and how a null is written in the CSV (e.g. an empty field rather than NA)
 - valid range or allowed values
 - a short description
- **Keys:**
 - a unique primary key for each table
 - foreign keys from `crash_locations` to the intersection or segment tables
- **Rules across columns and tables:**
 - severity categories sum to the total
 - serious crashes $\leq$ injury crashes

- crash-location counts sum to the site totals
 - every site referenced in `crash_locations` exists
- **When each column becomes known** (the leakage tags from earlier): this lets the ML module refuse to use an after-the-crash field as a feature.
- **Spatial and time conventions:**
 - CRS is EPSG:8059 and coordinates are in metres
 - the crash data window
 - which year the traffic volumes refer to
- **CSV format details:** UTF-8 encoding, comma delimiter, . as the decimal point, ISO dates, how booleans are written, header row required.
 - CSV loses type information, so the contract has to restore it explicitly. Otherwise "06672192063094" becomes a number and loses its leading zero, which breaks the join.
- **Version and change policy:**
 - Use semantic versioning. Adding a nullable column is a minor change; renaming, removing or redefining a column is a major change.
 - The ML module declares which major version it accepts, and refuses anything else.

Format options:

- **Frictionless Data Package (datapackage.json):** my recommendation. It is a standard designed for exactly this case: a set of CSVs, each with a schema, plus primary keys, foreign keys, constraints and units. You can add your own fields (leakage tags, CRS) as extra properties. It has a Python validator, and it is the format CKAN portals understand, which ties in with data.gov.au.
- **Pandera schemas exported to YAML:** convenient because you're already using Pandera for cleaning. However, foreign keys and links between tables are weaker.
- **Open Data Contract Standard (ODCS):** more of an organisational format, covering ownership, service-level agreements and support. That's overkill for one pipeline.

A good combination: the Frictionless file is the contract, and the ML module generates or loads matching Pandera schemas from it for checks within each table. Rules that span tables are written once as named checks that the contract refers to.

Publishing safely

The ML module must never read a half-written output, so publish in this order:

1. Write the three CSVs to a temporary folder.
2. Validate them against the contract on the pipeline side.
3. Write a **manifest** containing:
 - the contract version
 - the run ID and the source release hashes from the fetch stage
 - row counts per table
 - the SHA-256 hash of each CSV
4. Move the whole folder into place in one step. The manifest is written last, so it acts as the "complete" marker.

On the ML side, validation runs in this order:

1. The manifest exists, and its contract version is one the module accepts.
2. The file hashes and row counts match the manifest. This detects truncation or tampering.

3. Each table passes its schema.
4. The cross-table rules pass.
5. Only then is anything loaded for training.

If any step fails, the ML module keeps using the last good snapshot instead of training on bad data. This is the same principle as the fetch stage.

You — Sun 27 Sept 2026, 14:44

[attached file: assessment_requirements_safeguards.xlsx] these are all the things we discussed, can you create an automated pipeline that can fulfill each issue we talked about using python/python libraries?

Claude — 14:44

Actions taken

- Ran a command: Read all sheets of the requirements spreadsheet

The spreadsheet covers everything we discussed, so building this is straightforward. A few answers would change what I build, so I'll ask first.

Actions taken

- Asked you: This looks like an assessment brief, and earlier you asked me not to write the code. Does your course's AI policy allow AI-written code for this submission? | The brief says 'three site-level tables'. Which three? | Tamper detection: the brief says any hash mismatch stops the run, but a genuine new release always changes the hash. How should it behave?
- Your answer to *This looks like an assessment brief, and earlier you asked me not to write the code. Does your course's AI policy allow AI-written code for this submission?* → **Allowed, write it all**
- Your answer to *The brief says 'three site-level tables'. Which three?* → **Intersections, segments, crash links (Recommended)**
- Your answer to *Tamper detection: the brief says any hash mismatch stops the run, but a genuine new release always changes the hash. How should it behave?* → **Halt only on unexplained change (Recommended)**
- Ran a command: Test CKAN API access from the workspace
- Read web page: https://data.sa.gov.au/data/api/3/action/package_show?id=road-crashes-in-sa
- Read web page: https://data.sa.gov.au/data/api/3/action/package_search?q=traffic%20signals&rows=5
- Read web page: https://data.sa.gov.au/data/api/3/action/package_search?q=crossings%20OR%20%22traffic%20volume%22&rows=10&fq=organization:department-for-infrastructure-and-transport
- Read web page: https://data.sa.gov.au/data/api/3/action/package_search?q=pedestrian%20crossings&rows=6
- Read web page: https://data.sa.gov.au/data/api/3/action/package_search?q=traffic%20volumes&rows=6
- Ran a command: Profile attribute domains for schema design
- Ran a command: Create venv and resolve current library versions

Writing the package now. I'm starting with the shared pieces: errors, config and logging.

Actions taken

- Created file `sa_crash_pipeline/pipeline/errors.py`
- Created file `sa_crash_pipeline/config.toml`

- Edited file `sa_crash_pipeline/config.toml`
- Created file `sa_crash_pipeline/pipeline/config.py`
- Created file `sa_crash_pipeline/pipeline/net.py`
- Created file `sa_crash_pipeline/pipeline/ckan.py`
- Created file `sa_crash_pipeline/pipeline/archive.py`
- Created file `sa_crash_pipeline/pipeline/state.py`
- Created file `sa_crash_pipeline/pipeline/fetch.py`
- Edited file `sa_crash_pipeline/pipeline/fetch.py`
- Created file `sa_crash_pipeline/pipeline/readers.py`
- Ran a command: Test pandera failure-case output under pandas 3
- Created file `sa_crash_pipeline/pipeline/schemas.py`
- Edited file `sa_crash_pipeline/pipeline/schemas.py`
- Edited file `sa_crash_pipeline/pipeline/schemas.py`
- Created file `sa_crash_pipeline/pipeline/quarantine.py`
- Ran a command: Compute SA Lambert coordinate bounds for contract ranges
- Created file `sa_crash_pipeline/pipeline/transform.py`
- Edited file `sa_crash_pipeline/pipeline/transform.py`
- Ran a command: Replace NA identity checks with `pd.notna`

Now the data contract. I'm generating it once from a spec, so every field gets a unit, a role and a "known-at" tag.

Actions taken

- Ran a command: Generate the Frictionless data contract and check it loads
- Created file `sa_crash_pipeline/pipeline/contract.py`
- Created file `sa_crash_pipeline/pipeline/load.py`
- Created file `sa_crash_pipeline/pipeline/run.py`
- Created file `sa_crash_pipeline/data_ingestion.py`
- Created file `sa_crash_pipeline/pipeline/__init__.py`
- Edited file `sa_crash_pipeline/pipeline/fetch.py`
- Edited file `sa_crash_pipeline/pipeline/fetch.py`
- Created file `sa_crash_pipeline/validate_snapshot.py`

Now a first end-to-end run in local mode, using your five files zipped the way the portal ships them.

Actions taken

- Ran a command: Zip the uploaded datasets and run the pipeline in local mode
- Ran a command: Validate the snapshot, inspect outputs, and re-run to test skip
- Ran a command: Inspect raw CSV formatting, manifest and quarantine files
- Edited file `sa_crash_pipeline/pipeline/load.py`
- Created file `sa_crash_pipeline/tests/helpers.py`
- Ran a command: Rename settings helper and tidy a test geometry
- Created file `sa_crash_pipeline/tests/conftest.py`
- Created file `sa_crash_pipeline/tests/test_security_network.py`
- Created file `sa_crash_pipeline/tests/test_security_archive.py`
- Created file `sa_crash_pipeline/tests/test_tamper_and_pipeline.py`

- Created file `sa_crash_pipeline/tests/test_validation_contract.py`
- Ran a command: Run the test suite
- Ran a command: Relax `geom_type` dtype and rerun failing tests
- Ran a command: Make run IDs collision-proof and rerun all tests
- Ran a command: Use named custom checks and fix `argparse` exit handling
- Ran a command: Fix local-mode test and rerun suite

All 66 tests pass. One thing I can't test from here is whether the portal's real download links redirect to a storage host that isn't on the allow-list. I'll check that through the web tool.

Actions taken

- Read web page: https://data.sa.gov.au/data/dataset/cf8d8549-b976-4523-88ad-46dd32053696/resource/5fcbce14-ad30-4737-aa7d-5b1cfb939eb9/download/roadcrashes_2020_2024_geojson.zip
- Read web page: https://www.dptiapps.com.au/dataportal/TrafficSignals_geojson.zip
- Ran a command: Install runtime deps into a clean venv and freeze them
- Ran a command: Install pinned deps in a fresh venv, run tests and the real pipeline
- Ran a command: Pin `pytest` deps, add contract header, view quality report
- Created file `sa_crash_pipeline/README.md`
- Ran a command: Add fallback/allow-list tests, run suite and a live-mode failure check
- Ran a command: Allow proxy-resolved hosts and retest live and offline
- Ran a command: Check for runtime warnings and update `README` details
- Ran a command: Confirm every project file carries a header
- Ran a command: Clean generated folders and package the project
- Sent file: `sa_crash_pipeline.zip`

I've built the pipeline in `sa_crash_pipeline.zip`, and it covers every row of your requirements spreadsheet. On your five real datasets it runs end to end in about 8 seconds and exits with code 0. All 68 offline tests pass in a fresh environment set up only from the pinned requirements.

I couldn't test it against the live `data.sa.gov.au` and DIT servers, because this workspace can't reach them. The live run failed there and exited with 1 as it should, leaving the previous snapshot in place. If a real download redirects to a host that isn't on the allow-list, the run stops and names that host. Check the host, then add it to `allowed_hosts` in `config.toml`.

How it runs

- `python data_ingestion.py` does fetch → validate → transform → load in one go.
- `--source api` (the default) finds the current files through the CKAN API, `--source url` uses fixed links, and `--source local` reads zips you've downloaded yourself.
- The ML side calls `load_validated_snapshot()` from `pipeline/contract.py`, or runs `python validate_snapshot.py`.

Results on the 2020-2024 data

- **Tables:** 693 signalised intersections, 2,685 road segments and 43,522 crash locations (63,230 crashes).
- **Assignment:**
 - 11,291 crashes went to intersections.
 - 33,073 went to segments.

- 18,866 (about 30%) are unassigned, because the traffic-volume network only covers state-maintained roads, so crashes on local roads have no segment to snap to.
- **Quarantine:** three crossings had no ID, so they went to quarantine with the rule they broke.

Departures from the spreadsheet

- **Tamper detection:** as you chose, the run stops only on a change the publisher's metadata doesn't explain. An unexplained change is accepted only with `--approve-release <dataset>`. Any edit to a stored raw archive always stops the run.
- **Proxy networks:** on a network that sends traffic through a proxy, the check that the download server has a public address is skipped with a warning, because local DNS can fail there. The allow-list and certificate checks still apply.

Things to check before relying on the results (the README covers each one)

- **Carriageway "L":** 800 segments use this code. If their traffic volume covers only one direction, exposure on divided roads is off by a factor of 2. The DIT data dictionary should say which.
- **Signalised intersections only:** roundabouts and stop or give-way junctions aren't in any of the five datasets, so their crashes are counted on segments.
- **Missing minor-road volumes:** 349 intersections have no volume for the minor road, so their exposure undercounts the vehicles entering.

Every file header includes an AI-assistance line, since you said your course permits AI-written code with acknowledgement. Change the wording if your course's rules ask for something different.